

SoundHound CIE Interview Prep

SuperOwl Deep Dive

Cognitive Implementation Engineer I

Amelia Global Services, Bengaluru

Generated: June 2026

SuperOwl — Voice AI Call System (Complete Deep Dive)

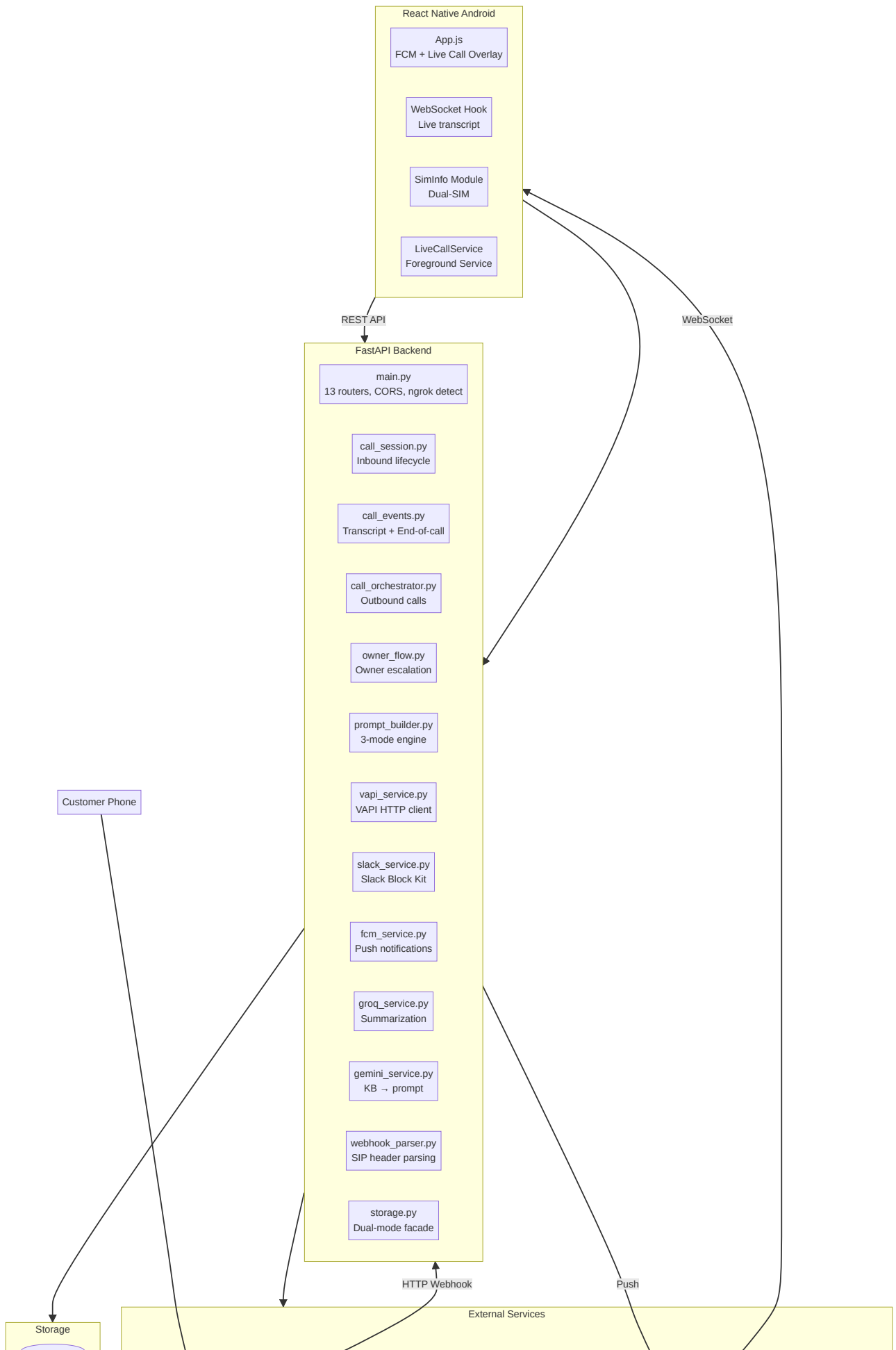
*****For****: Full internal understanding of every component you built.*

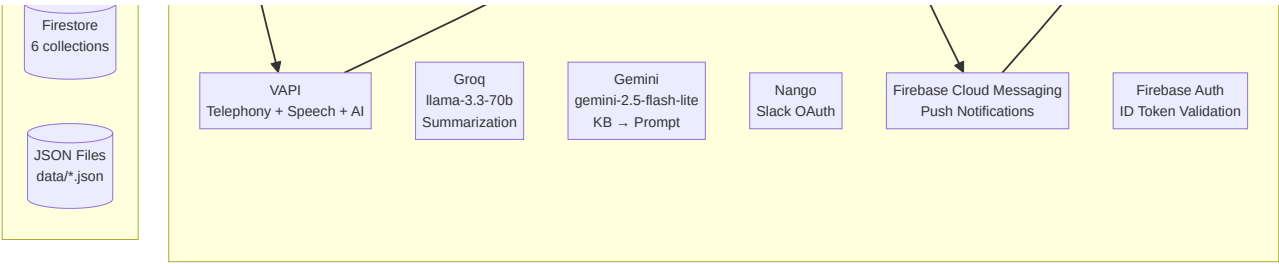
*****Goal****: Answer any question about how the system actually works — webhook payloads, storage layer, prompt building, escalation logic, WebSocket streaming, Slack integration, mobile app internals.*

What is SuperOwl?

A multi-tenant voice AI platform where businesses automate customer phone calls. Each business gets its own AI configuration. Customers call → AI answers → resolves or escalates. Owners monitor via Slack and mobile app. ---

High-Level Architecture





Tech Stack

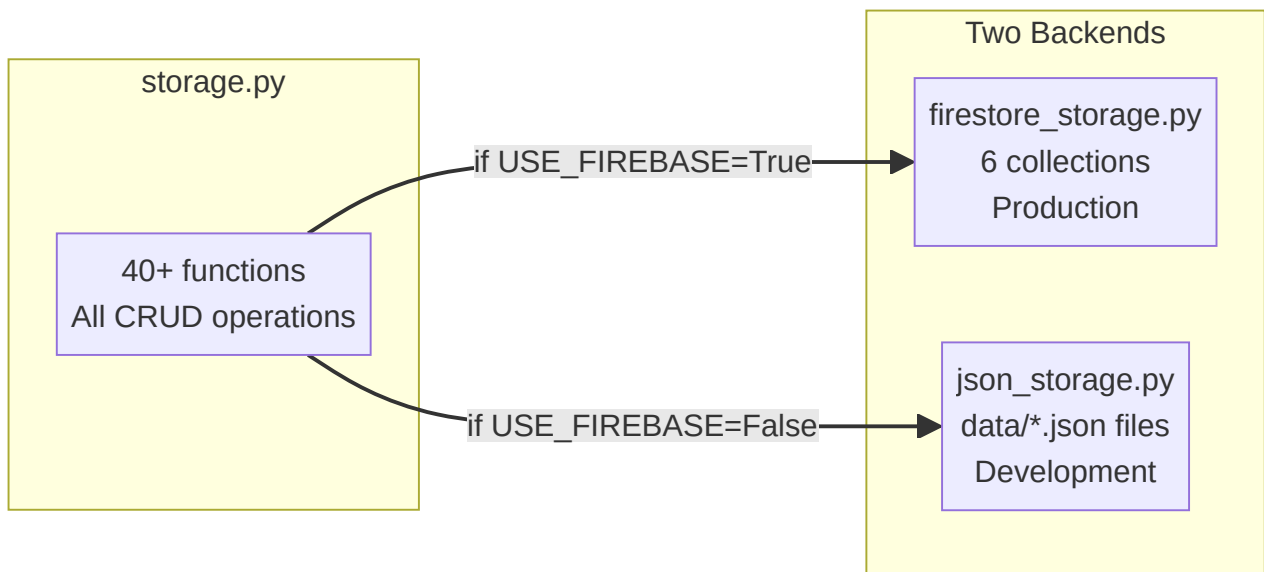
Component	Technology	Why
Backend	FastAPI (Python 3.12)	Async webhooks, WebSocket, auto docs
Voice AI	VAPI (api.vapi.ai)	Handles telephony, STT, AI conversation, TTS
Database	Firebase Firestore (prod) / JSON files (dev)	Real-time sync, serverless
LLM	Groq (llama-3.3-70b)	Transcript summarization
LLM	Gemini (gemini-2.5-flash-lite)	KB → business system prompt
Push	Firebase Cloud Messaging	Android push notifications
Auth	Firebase Auth	ID token validation
Slack	Nango OAuth proxy	Multi-tenant Slack integration
WebSocket	FastAPI WebSocket	Live transcript streaming
Mobile	React Native 0.73 (Android)	Dual-SIM via Kotlin native modules
Deployment	Docker → Cloud Run (asia-south1)	Auto-scaling, 512Mi, 1-3 instances

Project Structure — Every File Explained

```
`` superowl_mobile/ | — backend/ | | — main.py # Entry point | | — app/ | | | — core/ | | | | —
config.py # Pydantic Settings from .env | | | | — storage.py # Dual-mode storage facade | | | | —
firestore_storage.py # Firestore CRUD (40+ functions) | | | | — json_storage.py # JSON file CRUD with
advisory locking | | | | — logging_config.py # Structured JSON logging + request ID | | | — services/
| | | | — call_session.py # ★ Inbound call lifecycle (CRITICAL) | | | | — call_events.py # ★ Transcript
+ end-of-call processing | | | | — call_orchestrator.py # Outbound call creation | | | | —
owner_flow.py # ★ Owner escalation + SIP transfer | | | | — prompt_builder.py # ★ 3-mode prompt
engine | | | | — vapi_service.py # VAPI HTTP client | | | | — fcm_service.py # Firebase Cloud
Messaging | | | | — groq_service.py # Transcript summarization | | | | — gemini_service.py # KB →
business prompt generation | | | | — nango_service.py # Slack OAuth via Nango | | | | —
```

slack_service.py # Slack Block Kit notifications | | | └─ webhook_parser.py # SIP header + tool call parsing | | └─ routers/ | | └─ vapi_webhook.py # ★ Main webhook dispatcher | | └─ mobile.py # Mobile app API (profile, calls, whisper) | | └─ ws_live_call.py # ★ WebSocket live transcript | | └─ billing.py # Credit system | | └─ businesses.py # Business CRUD | | └─ call_logs.py # Call history | | └─ trigger.py # Outbound callback trigger | | └─ onboarding.py # Slack OAuth flow | | └─ slack_events.py # Slack Events API (whisper from thread) | | └─ slack_commands.py # Slash commands (/superowl-*) | | └─ slack_actions.py # Interactive buttons | | └─ playground.py # Dev/seed endpoints | | └─ prompts.py # Shared prompt template management | └─ scripts/ # Migration, setup, inspection | └─ data/ # JSON storage (dev mode) | └─ Dockerfile | └─ mobile/ | └─ App.js # Root: FCM handler + live call overlay | └─ src/ | └─ screens/ # 24 screens | └─ components/ # Shared UI | └─ services/ # useLiveCallSocket, useSimInfo | └─ state/ # Global app state (Context) | └─ navigation/ # Stack + bottom tabs | └─ functions/ # Firebase Cloud Functions (minimal) └─ root scripts # VAPI debugging, log fetching ``` ---

The Storage Layer (Dual-Mode)



storage.py — The Facade

40+ functions that all call either `firestore_storage` or `json_storage` based on the `USE_FIREBASE` env var. This is the ONLY file that touches storage. All services go through this facade. ```python
 get_user_profile(uid) # Get user profile with nested business
 get_user_profile_by_phone(phone) # Lookup by phone (for ANI resolution)
 update_user_profile(uid, data) # Update profile fields
 create_call_log(data) # Create a call record
 update_call_log(call_id, data) # Update call status, transcript, summary
 get_call_logs(uuid) # Get all calls for a business
 get_call_log_by_vapi_id(vapi_id) # Lookup by VAPI call ID
 save_session(session_id, data) # Save call session data
 get_session(session_id) # Get session data ```

firestore_storage.py — Production

6 Firestore collections:

Collection	Key Fields	Purpose
<code>user_profiles</code>	<code>uid, name, phone, fcm_token, mode, credits, features, business{}</code>	Unified profile + nested business config
<code>user_profiles/{uid}/sims</code>	<code>simlccld, carrier, phone_number</code>	SIM card registry
<code>call_logs</code>	<code>uuid, call_type, vapi_call_id, customer_phone, duration, outcome, transcript, summary, slack_ts</code>	Call history
<code>sessions</code>	<code>session_id, state, active_call_id</code>	Active call sessions
<code>prompt_templates</code>	<code>shared_system_prompt</code>	Global prompt template
<code>credit_transactions</code>	<code>uid, amount, plan, timestamp</code>	Billing history

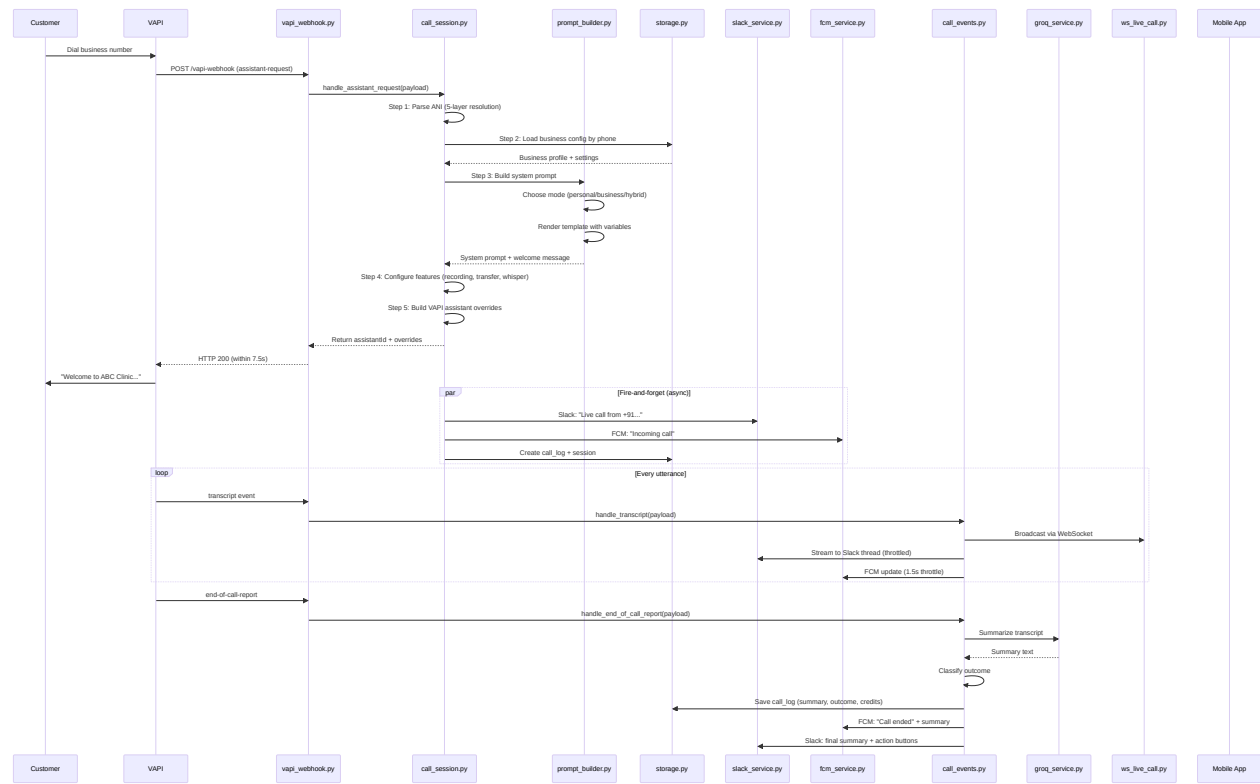
json_storage.py — Development

Uses `fcntl.flock` advisory file locking for atomic writes. Each collection maps to a JSON file in `data/` :

```
`` data/user_profiles.json data/call_logs.json data/sessions.json data/prompts.json data/kb/{uuid}.txt data/phone_map.json `` ---
```

Inbound Call Flow — Complete Step-by-Step

This is the CRITICAL path. Understand every step.



Step 1: VAPI Webhook Arrives (`vapi\_webhook.py`)

```
python
```

vapi_webhook.py — The dispatcher

```
@router.post("/vapi-webhook") async def vapi_webhook(request: Request): payload = await request.json()
event_type = payload.get("message", {}).get("type") if event_type == "assistant-request": await
handle_assistant_request(payload) elif event_type == "transcript": await handle_transcript(payload) elif
event_type == "end-of-call-report": await handle_end_of_call_report(payload) elif event_type == "hang":
await handle_hang(payload) elif event_type == "tool-call": tool_name = payload["message"]["toolCall"]
["name"] if tool_name == "notify_owner": await handle_notify_owner(payload) elif tool_name ==
"owner_decision": await handle_owner_decision(payload) return {"ok": True} ``
```

Step 2: ANI Resolution (`call\_session.py`)

```
``python
```

call_session.py — 5-layer ANI resolution

```
async def handle_assistant_request(payload): call = payload["call"] caller_phone = None # Layer 1:
Diversions SIP Header # Some carriers add "Diversions" header with original caller # Format:
sip:9999999999@sip.vapi.ai diversion = call.get("sipHeaders", {}).get("Diversions") if diversion: caller_phone
= parse_sip_uri(diversion) # Extract "9999999999" # Layer 2: SIP From/To Headers if not caller_phone:
caller_phone = call.get("from") # VAPI's normalized caller number # Layer 3: Phone Map (in-memory
cache) # Pre-populated when businesses register their phone numbers # Key: phone_number → Value:
business_uuid if not caller_phone: business_uuid = phone_map.get(call.get("to")) # Layer 4: DB Lookup by
Business Phone if not business_uuid: business_uuid = await
storage.get_business_by_phone(call.get("to")) # Layer 5: DB Lookup by Owner Phone if not
business_uuid: business_uuid = await storage.get_business_by_owner_phone(caller_phone) # Now load
full business config profile = await storage.get_user_profile(business_uuid) business = profile["business"] #
... continue to build assistant config ``
```

Step 3: Build Assistant Config (`prompt\_builder.py`)

```
``python
```

prompt_builder.py — 3-mode prompt engine

```
def build_system_prompt(business, mode): """ Three modes:
```

- PERSONAL: Owner's personal assistant
- BUSINESS: Business representative
- HYBRID: Detect intent first, then route

```
"""
```

```
if mode == "personal": return PERSONAL_PROMPT.format( owner_name=business["owner_name"],
greeting=business.get("greeting_message", "Hi, I'm your assistant."),
spam_rules=business.get("spam_filter_rules", "Block telemarketers.") ) elif mode == "business": return
BUSINESS_PROMPT.format( business_name=business["display_name"],
hours=business.get("business_hours", "Not specified"), services=business.get("services", "Not specified"),
faq=business.get("faq", ""), booking_link=business.get("booking_link", "") ) elif mode == "hybrid": return
HYBRID_PROMPT.format( business_name=business["display_name"],
owner_name=business["owner_name"], # Includes intent detection instructions: # "If caller asks about the
business, use business flow." # "If caller asks for the owner personally, use personal flow." # "If unclear,
ask: 'Is this about {business_name} or {owner_name}?'") ) ````
```

Step 4: Configure Features

```
```python
```

## Features are per-business toggles

---

```
features = business.get("features", {}) assistant_overrides = { "model": { "provider": "openai", "model": "gpt-4o-mini", "systemPrompt": system_prompt, }, "voice": { "provider": "11labs", # or other provider "voiceId": business.get("voice_id", "default"), }, "transcriber": { "provider": "deepgram", "language": business.get("language", "en"), }, "recordingEnabled": features.get("callRecording", False), "endCallEnabled": features.get("endCall", True), "transferCallEnabled": features.get("callTransfer", True), }
```

## Add tools based on features

---

```
tools = [] if features.get("callTransfer"): tools.append({ "type": "function", "function": { "name":
"notify_owner", "description": "Transfer call to owner or request human help", "parameters": { "type":
"object", "properties": { "reason": { "type": "string" } } } }) ``
```

### Step 5: Return to VAPI (within 7.5 seconds)

```
``python
```

## Return assistantId to VAPI

---

```
return { "assistantId": business.get("vapi_assistant_id"), "assistantOverrides": assistant_overrides } ``
```

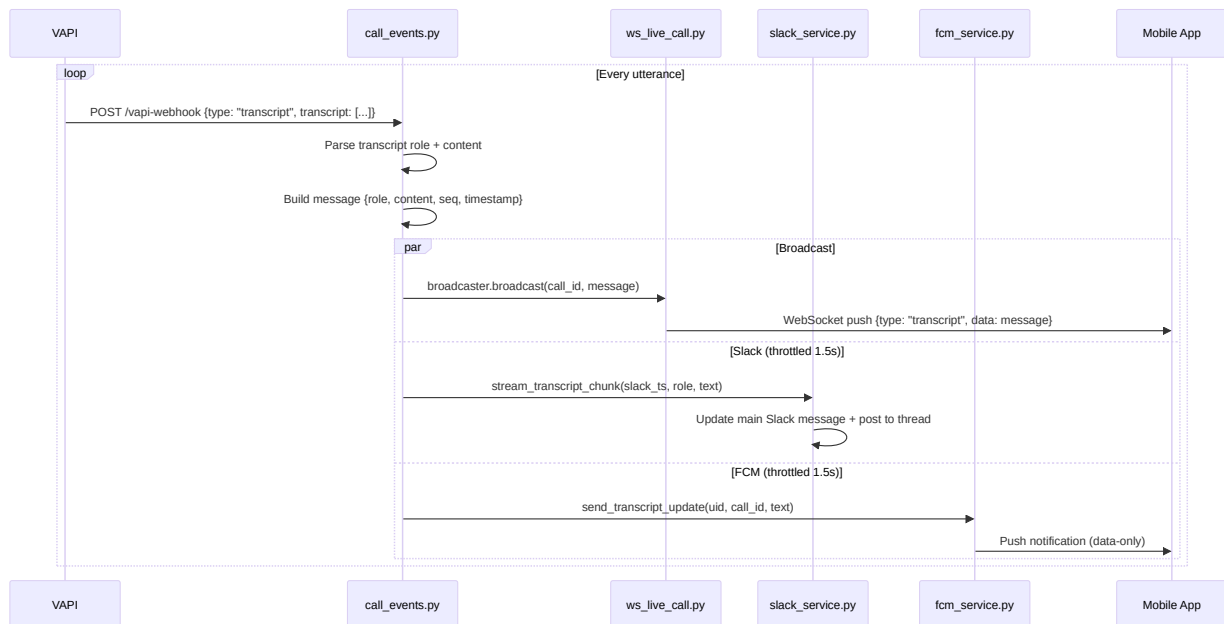
### Step 6: Fire-and-Forget Notifications

```
``python
```

# After responding to VAPI, do non-critical work asynchronously

```
async def post_call_setup(business, call_id, caller_phone): # These run in parallel, but don't block the
webhook response asyncio.create_task(slack_service.send_live_call_notification(business=business,
call_id=call_id, caller_phone=caller_phone)) asyncio.create_task(fcm_service.send_to_user(
uid=business["owner_uid"], data={ "type": "incoming_call", "call_id": call_id, "caller": caller_phone })) #
Create call log in storage await storage.create_call_log({ "uuid": business["uuid"], "call_type": "inbound",
"vapi_call_id": call_id, "customer_phone": caller_phone, "status": "in_progress", "created_at":
datetime.now().isoformat() }) `` ---
```

## Transcript Streaming Flow



## Inside `call\_events.py` — `handle\_transcript`

```
python async def handle_transcript(payload): call_id = payload["call"]["id"] transcript = payload["call"]
["transcript"] # Get the last message from the transcript if not transcript: return last_msg = transcript[-1] role
= last_msg["role"] # "user" or "assistant" content = last_msg["message"] # The actual text # Find the call log
to get business info call_log = await storage.get_call_log_by_vapi_id(call_id) business = await
storage.get_business_by_uuid(call_log["uuid"]) # Broadcast via WebSocket (immediate, no throttle) await
ws_broadcaster.broadcast(call_id, { "type": "transcript", "role": role, "content": content, "seq": len(transcript),
"timestamp": datetime.now().isoformat() }) # Update call log in storage await
storage.update_call_log(call_log["call_id"], { "transcript": transcript # Store full transcript array }) # Slack
(throttled — check last update time) if time_since_last_slack_update() > 1.5: # seconds await
slack_service.stream_transcript_chunk(slack_ts=call_log.get("slack_ts"), role=role, text=content) # FCM
```

```
(throttled — check last update time) if time_since_last_fcm_update() > 1.5: # seconds await
fcm_service.send_to_user(uid=business["owner_uid"], data={"type": "transcript_update", "call_id": call_id,
"text": content}) ``` ---
```

## End-of-Call Processing

```
```python async def handle_end_of_call_report(payload): call_id = payload["call"]["id"] transcript =
payload["call"]["transcript"] duration = payload["call"]["duration"] # seconds ended_reason = payload["call"]
["endedReason"] # 1. Summarize with Groq summary = await
groq_service.summarize_transcript(transcript) # 2. Classify outcome outcome =
classify_outcome(summary, ended_reason) # "resolved" | "needs_followup" | "escalated" | "incomplete" #
3. Update call log await storage.update_call_log(call_id, { "status": "completed", "duration": duration,
"ended_reason": ended_reason, "summary": summary, "outcome": outcome, "completed_at":
datetime.now().isoformat() }) # 4. Deduct credits credits_used = calculate_credits(duration) await
storage.deduct_credits(business_uuid, credits_used) # 5. Close WebSocket session await
ws_broadcaster.close_session(call_id) # 6. FCM: call ended notification await
fcm_service.send_to_user(uid=owner_uid, data={ "type": "call_ended", "call_id": call_id, "duration":
duration, "outcome": outcome, "summary": summary }) # 7. Slack: final summary with action buttons await
slack_service.mark_call_completed( slack_ts=call_log["slack_ts"], duration=duration, outcome=outcome,
summary=summary ) ```
```

Groq Summarization

```
```python
```

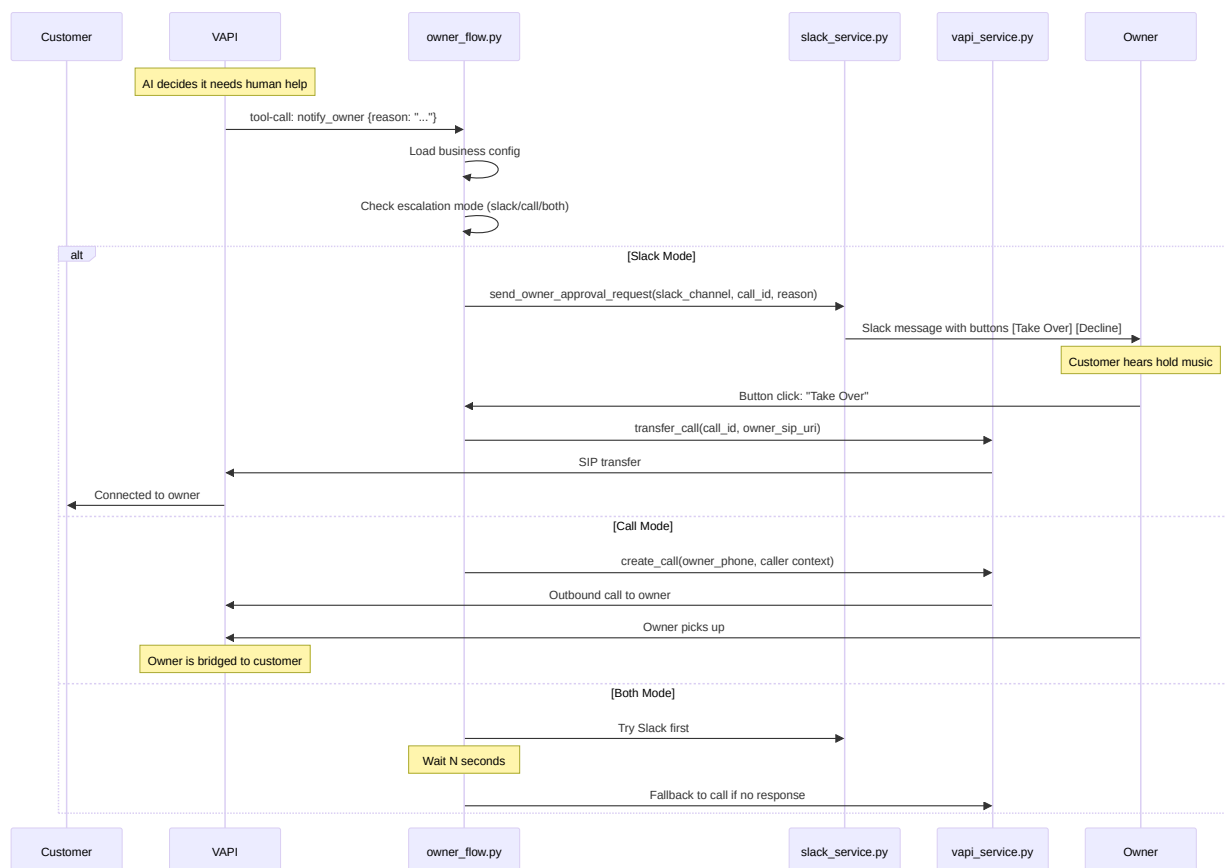
# groq\_service.py

async def summarize\_transcript(transcript): """ Takes full transcript array, produces concise summary.  
Model: llama-3.3-70b-versatile via Groq """ # Format transcript for the LLM formatted = "\n".join([ f"  
{msg['role']}: {msg['message']}" for msg in transcript ]) prompt = f"""Summarize this customer call in 2-3  
sentences:

- What did the customer want?
- Was it resolved?
- Any action needed?

Transcript: {formatted}""" response = await groq\_client.chat.completions.create( model="llama-3.3-70b-versatile", messages=[{"role": "user", "content": prompt}], max\_tokens=200 ) return  
response.choices[0].message.content ``` ---

## Owner Escalation Flow



## Inside `owner\_flow.py`

```
```python async def handle_notify_owner(payload): tool_call = payload["message"]["toolCall"] reason =  
tool_call["arguments"]["reason"] call_id = payload["call"]["id"] call_log = await
```

```

storage.get_call_log_by_vapi_id(call_id) business = await storage.get_business_by_uuid(call_log["uuid"])
escalation_mode = business.get("escalation_mode", "slack") if escalation_mode in ("slack", "both"): # Send
Slack approval request with interactive buttons slack_ts = await
slack_service.send_owner_approval_request( channel=business["slack_channel_id"], call_id=call_id,
reason=reason, caller_phone=call_log["customer_phone"] ) # Store slack_ts for later reference await
storage.update_call_log(call_log["call_id"], { "slack_approval_ts": slack_ts }) if escalation_mode == "call": #
Direct call to owner await vapi_service.create_call( to=business["owner_no"], from_=business["phone"],
assistant_id=business["vapi_assistant_id"], metadata={"type": "owner_escalation", "original_call_id":
call_id} ) # Tell VAPI to play hold music while waiting return {"result": "Owner notified. Playing hold music."}
...

```

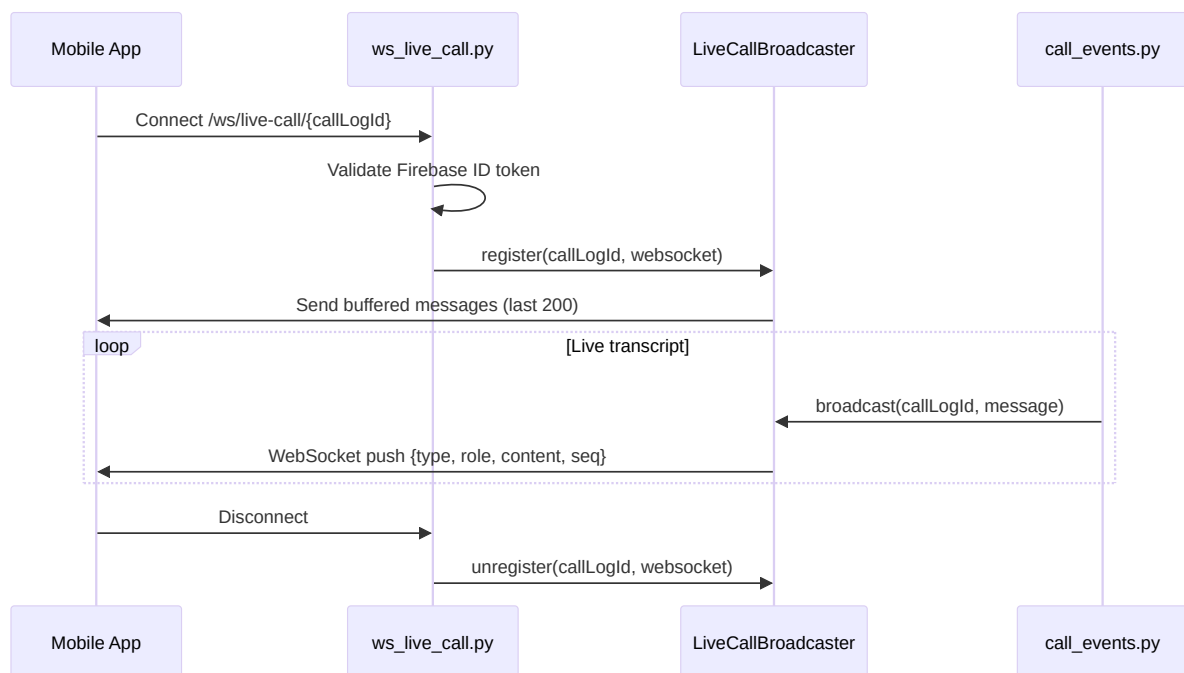
Owner Decision Handling

```

python async def handle_owner_decision(payload): tool_call = payload["message"]["toolCall"] decision =
tool_call["arguments"]["decision"] # "yes" or "no" call_id = payload["call"]["id"] call_log = await
storage.get_call_log_by_vapi_id(call_id) business = await storage.get_business_by_uuid(call_log["uuid"]) if
decision == "yes": # Transfer call to owner's phone owner_sip = f"sip:{business['owner_no']}@vapi.ai" await
vapi_service.transfer_call(call_id, owner_sip) await storage.update_call_log(call_log["call_id"], { "outcome":
"escalated_to_owner" }) elif decision == "no": # Decline — play polite message await
vapi_service.send_message(call_id, { "message": "I'm sorry, I wasn't able to reach anyone right now. "
"Can I take a message and have someone call you back?" }) await
storage.update_call_log(call_log["call_id"], { "outcome": "declined_by_owner" })

```

WebSocket Live Call (`ws\_live\_call.py`)



LiveCallBroadcaster Class

```
```python class LiveCallBroadcaster: def __init__(self): self.connections = {} # call_id → set of WebSocket connections self.buffers = {} # call_id → deque of last 200 messages async def register(self, call_id, websocket): if call_id not in self.connections: self.connections[call_id] = set() self.buffers[call_id] = deque(maxlen=200) self.connections[call_id].add(websocket) # Send buffered history for catch-up for msg in self.buffers[call_id]: await websocket.send_json(msg) async def broadcast(self, call_id, message): # Store in buffer if call_id in self.buffers: self.buffers[call_id].append(message) # Send to all connected clients if call_id in self.connections: dead = set() for ws in self.connections[call_id]: try: await ws.send_json(message) except: dead.add(ws) self.connections[call_id] -= dead async def close_session(self, call_id): # Send close message, then clean up if call_id in self.connections: for ws in self.connections[call_id]: await ws.send_json({"type": "call_ended"}) self.connections.pop(call_id, None) self.buffers.pop(call_id, None) ```
```

## Client-Side Reconnection (`useLiveCallSocket.js`)

```
```javascript // useLiveCallSocket.js const useLiveCallSocket = (callLogId) => { const [transcripts, setTranscripts] = useState([]); const [status, setStatus] = useState('connecting'); useEffect(() => { let ws; let retryDelay = 1000; const connect = () => { ws = new WebSocket( ws://host/ws/live-call/${callLogId} ); ws.onopen = () => { setStatus('connected'); retryDelay = 1000; // Reset on success }; ws.onmessage = (event) => { const data = JSON.parse(event.data); if (data.type === 'transcript') { setTranscripts(prev => [...prev, data]); } else if (data.type === 'call_ended') { setStatus('completed'); } }; ws.onclose = () => { setStatus('disconnected'); // Exponential backoff reconnect setTimeout(connect, retryDelay); retryDelay = Math.min(retryDelay * 2, 30000); }; }; connect(); return () => ws?.close(); }, [callLogId]); return { transcripts, status }; }; ``` ---
```

Mobile App Architecture

App.js — Root Component

```
```javascript // App.js — The root const App = () => { // 1. FCM handler (always listening) useEffect(() => { const unsubscribe = messaging().onMessage(async (remoteMessage) => { const { type, call_id, ...data } = remoteMessage.data; if (type === 'incoming_call') { // Show overlay: "Incoming call from +91..." setActiveCall({ call_id, ...data }); } if (type === 'transcript_update') { // Update live transcript display addTranscript(data); } if (type === 'call_ended') { // Show summary notification showCallSummary(data); setActiveCall(null); } }); return unsubscribe; }, []); // 2. Live call overlay (when active) // Shows: transcript, whisper input, end call button, transfer button // 3. Navigation: Stack + Bottom Tabs // Tabs: Feed, Search, Schedule, Profile } ```
```

### FeedScreen.js — Call Log Display

```
```javascript // FeedScreen.js const FeedScreen = () => { const [calls, setCalls] = useState([]); const [filter, setFilter] = useState('all'); // all | calls | searches | chats useEffect(() => { // Fetch from backend api.getCallLogs(profile.uuid).then(setCalls); }, []); // Merge backend call logs with local notifications const mergedFeed = mergeByTimestamp(calls, notifications); // Filter logic const filtered = mergedFeed.filter(item => { if (filter === 'calls') return item.type === 'inbound' || item.type === 'outbound'; if (filter === 'searches')
```

```
return item.type === 'search'; return true; }); return ( ( navigation.navigate('CallDetail', { id: item.id })}
actions=[ { label: 'Call Back', onPress: () => triggerCallback(item) }, { label: 'View Transcript', onPress: () =>
viewTranscript(item) }, ] /> )} /> ); }; ``
```

LiveInboundScreen.js — Real-Time Call Console

```
``javascript // LiveInboundScreen.js const LiveInboundScreen = ({ callId }) => { const { transcripts, status }
= useLiveCallSocket(callId); const [whisperText, setWhisperText] = useState(""); // Controls const
handleWhisper = () => { api.sendWhisper(callId, whisperText); setWhisperText(""); }; const handleEndCall =
() => { api.endCall(callId); }; const handleTransferToOwner = () => { api.transferToOwner(callId); }; return (
{/ Live transcript /} ( ) /> {/ Controls /}   
```

```
); }; ``
```

CallForwardingScreen.js — Dual-SIM MMI Codes

```
``javascript // CallForwardingScreen.js const CallForwardingScreen = () => { const { simInfo } = useSimInfo();  
// Kotlin native module // MNC-based carrier classification const carrier = classifyCarrier(simInfo.mnc); // Jio:  
701, Airtel: 702, Vi: 703, BSNL: 704 // Generate carrier-specific MMI code const getForwardingCode =  
(carrier, superowlNumber) => { switch (carrier) { case 'jio': return `21${superowlNumber}#` ; case 'airtel': return  
`*21${superowlNumber}#` ; case 'vi': return `21${superowlNumber}#` ; case 'bsnl': return  
`21${superowlNumber}#` ; } }; const handleForward = () => { const code = getForwardingCode(carrier,  
businessPhone); Linking.openURL( `tel:${code}` ); // Opens dialer with MMI code }; return ( Carrier: {carrier}  
(SIM {simInfo.slot + 1}) Forward all calls to: {businessPhone}
```

```
); }; ``` ---
```

Slack Integration — Complete

slack_service.py — Block Kit Notifications

```
```python
```

## slack\_service.py

```
async def send_live_call_notification(business, call_id, caller_phone): """Send call start notification with action buttons""" blocks = [{ "type": "header", "text": { "type": "plain_text", "text": "📞 Live Call" } }, { "type": "section", "fields": [{ "type": "mrkdwn", "text": f"Caller: \n{caller_phone}" }, { "type": "mrkdwn", "text": f"Business: \n{business[\"display_name\"]}" }, { "type": "mrkdwn", "text": f"Time: \n{datetime.now().strftime(\"%H:%M\")}" }] }, { "type": "actions", "elements": [{ "type": "button", "text": { "type": "plain_text", "text": "Take Over" }, "style": "primary", "action_id": "takeover", "value": call_id }, { "type": "button", "text": { "type": "plain_text", "text": "End Call" }, "style": "danger", "action_id": "end_call", "value": call_id }, { "type": "button", "text": { "type": "plain_text", "text": "Whisper" }, "action_id": "whisper", "value": call_id }] }] response = await slack_client.chat.postMessage(channel=business["slack_channel_id"], blocks=blocks, text=f"Live call from {caller_phone}") return response["ts"] # Store this as slack_ts for thread updates async def stream_transcript_chunk(slack_ts, role, text): """Update main message + post to thread""" # Post to thread await slack_client.chat.postMessage(channel=channel, thread_ts=slack_ts, text=f"{role}: {text}") async def mark_call_completed(slack_ts, duration, outcome, summary): """Update main message with final summary""" await slack_client.chat.update(channel=channel, ts=slack_ts, blocks=[{ "type": "section", "text": { "type": "mrkdwn", "text": f"✅ Call Ended\nDuration: {duration}s\nOutcome: {outcome}\nSummary: {summary}" } }])
```

### slack\_events.py — Whisper via Thread

```
```python
```

slack_events.py — When someone types in a Slack thread, it becomes a whisper

```
@router.post("/slack/events") async def slack_events(request: Request): payload = await request.json() # URL verification challenge if payload.get("type") == "url_verification": return {"challenge": payload["challenge"]} # Thread message = whisper event = payload.get("event", {}) if event.get("type") == "message" and event.get("thread_ts"): # Find the call associated with this thread call_log = await storage.get_call_by_slack_ts(event["thread_ts"]) if call_log and call_log["status"] == "in_progress": # Send whisper to VAPI await vapi_service.send_message( call_log["vapi_call_id"], {"message": event["text"], "type": "whisper"} )
```

slack_commands.py — Slash Commands

```
```python
```

## slack\_commands.py

```
@router.post("/slack/commands") async def slack_commands(request: Request): form = await request.form() command = form["command"] text = form["text"] channel = form["channel_id"] # Verify Slack signature if not verify_slack_signature(request): return {"text": "Invalid signature"} if command == "/superowl-whisper": # Find active call in this channel call = await storage.get_active_call_by_channel(channel) if call: await
```

```
vapi_service.send_message(call["vapi_call_id"], { "message": text, "type": "whisper" }) return {"text":
f"Whispered: {text}"} return {"text": "No active call found"} elif command == "/superowl-end": call = await
storage.get_active_call_by_channel(channel) if call: await vapi_service.end_call(call["vapi_call_id"]) return
{"text": "Call ended"} return {"text": "No active call found"} elif command == "/superowl-transfer": call = await
storage.get_active_call_by_channel(channel) if call: await vapi_service.transfer_call(call["vapi_call_id"], f"sip:
{text}@vapi.ai") return {"text": f"Transferring to {text}"} return {"text": "No active call found"} ```
```

## slack\_actions.py — Button Clicks

```
```python
```

slack_actions.py

```
@router.post("/slack/actions") async def slack_actions(request: Request): payload = json.loads((await
request.form()))["payload"] action = payload["actions"][0] action_id = action["action_id"] call_id =
action["value"] if action_id == "takeover": # Transfer call to owner's SIP call = await
storage.get_call_log_by_id(call_id) business = await storage.get_business_by_uid(call["uid"]) owner_sip =
f"sip:{business['owner_no']}@vapi.ai" await vapi_service.transfer_call(call["vapi_call_id"], owner_sip) return
{"text": "Call transferred to you"} elif action_id == "decline_transfer": await
vapi_service.send_message(call_id, { "message": "I'm sorry, no one is available right now." }) return {"text":
"Transfer declined"} elif action_id == "end_call": await vapi_service.end_call(call_id) return {"text": "Call
ended"} elif action_id == "whisper": # Open modal for whisper input return {"text": "Type your whisper in the
thread below"} ```
```

Billing System

```
```python
```

## billing.py

```
PLANS = { "starter": { "credits": 1000, "price": 999, "value": { "credits": 2200, "price": 1999, "power": { "credits":
4800, "price": 3499, "pro": { "credits": 10000, "price": 6999, } } }, @router.get("/balance/{uid}") async def
get_balance(uid: str): profile = await storage.get_user_profile(uid) return {"credits": profile.get("credits", 0)}
@router.post("/purchase/{uid}") async def purchase(uid: str, plan: str): plan_data = PLANS[plan] await
storage.update_user_profile(uid, { "credits": storage.get_user_profile(uid)["credits"] + plan_data["credits"] })
await storage.create_transaction(uid, plan, plan_data["price"]) return {"success": True, "credits_added":
plan_data["credits"]} @router.post("/deduct/{uid}") async def deduct(uid: str, call_id: str): # Firebase auth
required call = await storage.get_call_log_by_id(call_id) duration_seconds = call["duration"] credits_used =
calculate_credits(duration_seconds) profile = await storage.get_user_profile(uid) new_balance =
profile["credits"] - credits_used await storage.update_user_profile(uid, {"credits": max(0, new_balance)})
return {"credits_deducted": credits_used, "remaining": max(0, new_balance)} ```
```

## Gemini Service — KB → Prompt

```
```python
```

gemini_service.py

```
async def generate_business_prompt(knowledge_base_text): """ Takes raw KB text (business description,
services, FAQ) and generates a structured system prompt for the AI. Model: gemini-2.5-flash-lite """ prompt =
f"""You are a prompt engineering assistant. Given this business information, create a structured system
prompt for a voice AI assistant that handles customer calls. Business Information: {knowledge_base_text}
Generate a system prompt with these sections:
```

1. Role and tone
 2. Business knowledge (services, hours, location)
 3. Common scenarios and how to handle them
 4. Escalation rules (when to transfer to human)
 5. Greeting message
- Output in plain text, ready to use as a system prompt. """ response = await
gemini_client.generate_content(prompt) return response.text """ ---

Known Issues (From Code Audit)

Issue	Severity	Location	Fix
<code>get_user_profile_by_phone()</code> doesn't exist in <code>json_storage</code>	Critical	<code>json_storage.py:176</code>	Implement the function
<code>voice_id</code> stored but never sent to VAPI	Medium	<code>call_session.py</code>	Add to <code>assistantOverrides</code>
Multiple mobile buttons dead (Cancel Call, End Call)	Medium	mobile screens	Wire up event handlers
Hardcoded keystore password	Security	<code>build.gradle</code>	Move to env var
<code>showIntent</code> variable never set to true	Low	<code>FeedScreen.js</code>	Wire intent display

Architecture Decisions

1. **Dual storage (Firestore/JSON):** Production uses Firestore. Dev uses JSON with `fcntl.flock`. The facade (`storage.py`) routes transparently. Enables zero-setup local development.
 1. **Fire-and-forget for notifications:** VAPI has 7.5s webhook timeout. Slack/FCM can't be called within that window reliably. Solution: return to VAPI immediately, then send notifications as background tasks.
 1. **WebSocket + FCM dual channel:** WebSocket for high-frequency transcript (needs ordering). FCM for state changes (call started, ended). Different channels for different data patterns.
 1. **5-layer ANI resolution:** Real-world SIP headers are messy. Different carriers provide different headers. The 5-layer approach handles edge cases: Diversion header, SIP From/To, phone map cache, DB by business phone, DB by owner phone.
 1. **Multi-tenant via nested business object:** Business config lives inside `user_profiles`. No separate businesses collection. Denormalized for fast reads — one document fetch gets everything needed for call handling.

